

Playwright Test Agents

Official Documentation: <https://playwright.dev/docs/test-agents>

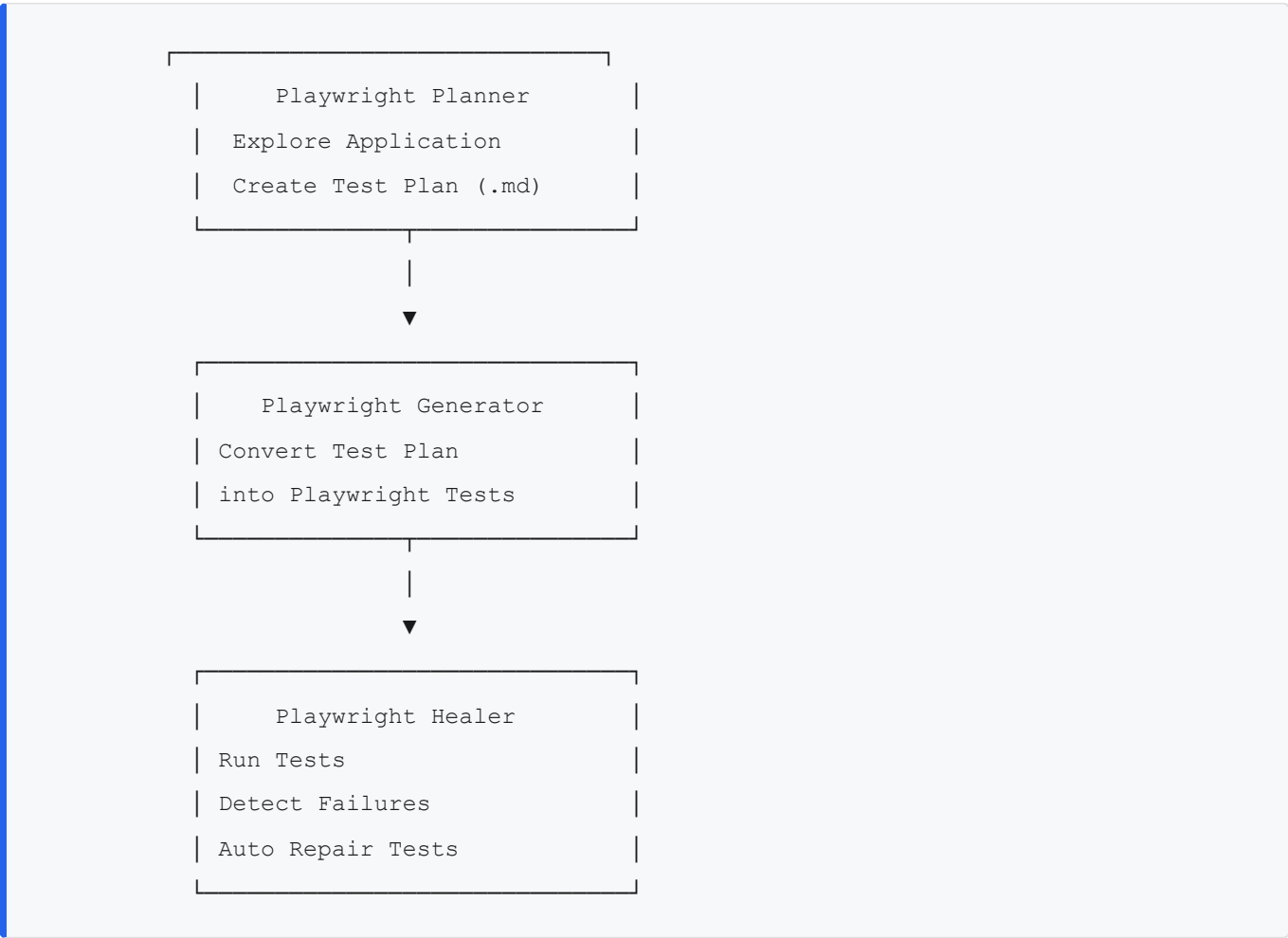
Introduction

Playwright Test Agents are AI-powered assistants introduced in Playwright that automate the complete test automation workflow. Instead of manually creating, maintaining, and fixing test scripts, these agents perform the work using natural language prompts.

Playwright provides **three built-in Test Agents**:

- **Planner** – Explores the application and creates a test plan.
- **Generator** – Converts the test plan into Playwright test scripts.
- **Healer** – Executes the tests and automatically repairs failing tests.

Playwright Test Agent Workflow



Environment Setup

Prerequisites

Before using Playwright Test Agents, make sure the following software is installed.

1. Install Visual Studio Code

Install the latest version of Visual Studio Code.

2. Enable GitHub Copilot

Playwright Test Agents currently work inside **VS Code** using **GitHub Copilot**.

3. Install Playwright

Install Playwright along with the required browsers.

```
npm init playwright@latest
```

4. Install Playwright Test Agents

Run the following command inside your Playwright project:

```
npx playwright init-agents --loop=vscode
```

After the installation completes, Playwright automatically:

- Installs the Test Agents
- Installs the **Playwright MCP Server**
- Creates the required configuration files
- Updates the project structure

Agent 1 – Playwright Test Planner

Purpose

The **Planner Agent** explores your web application and generates a detailed test plan in Markdown format.

Instead of manually writing test scenarios, the AI understands the application and creates them automatically.

Example Prompt

Select the agent:

```
playwright-test-planner
```

Prompt:

```
Explore the user interface of the web application at
```

```
https://www.saucedemo.com
```

```
Based on your exploration, create a test plan covering the five core user operations  
that a typical end user would perform on this e-commerce demo site.
```

Alternative Prompt

Instead of exploring the application, you can provide a Product Requirements Document (PRD).

Example:

```
Analyze the attached PRD and generate a complete Playwright test plan.
```

This approach is useful when the application is still under development.

Output

The Planner creates a Markdown test plan.

Example:

```
sauce-demo-core-user-operations.plan.md
```

The generated test plan typically includes scenarios such as:

- User Login
- Product Browsing
- Add Products to Cart
- Cart Review and Modification
- Checkout Process
- Logout
- Reset Application State

Agent 2 – Playwright Test Generator

Purpose

The **Generator Agent** converts the Markdown test plan into executable Playwright test scripts.

It automatically generates:

- Playwright test files
- Assertions
- Locators
- Test structure
- Organized test cases

Example Prompt

Select the agent:

```
playwright-test-generator
```

Prompt 1 : To generate tests for all the scenarios from the test plan:

```
Generate Playwright tests for all the scenarios provided in the test plan.
```

Prompt 2 : To generate test for the specific scenario from the test plan:

```
Create a Playwright test for scenario 1.1 from test-plan.md using the Playwright test g  
enerator agent.
```

Prompt 3 : Generate Tests Using Page Object Model (POM):

Generate Playwright tests for all the test scenarios provided in the test plan.

Requirements:

- Use TypeScript.
- Follow the Page Object Model (POM) design pattern.
- Create a separate Page Object class for each application page.
- Keep page locators and reusable page actions inside the Page Object classes.
- Keep test assertions and business scenarios inside the test (.spec.ts) files.
- Use Playwright's built-in locators (getByRole, getByLabel, getByText, getByPlaceholder, etc.) whenever possible.
- Avoid XPath unless absolutely necessary.
- Write clean, readable, and maintainable code.
- Organize the generated files using the following structure:

```
pages/  
    LoginPage.ts  
tests/  
    login.spec.ts
```

- Reuse Page Object methods across multiple tests.
- Generate meaningful method names and comments where appropriate.
- Follow Playwright and TypeScript best practices.

Output

The Generator creates Playwright test files, for example:

```
tests/
|
├─ login.spec.ts
├─ products.spec.ts
├─ cart.spec.ts
├─ checkout.spec.ts
└─ logout.spec.ts
```

These test files are ready to execute with minimal or no manual changes.

Agent 3 – Playwright Test Healer

Purpose

The **Healer Agent** runs the generated tests, identifies failures, and automatically attempts to repair them.

Common issues it can resolve include:

- Broken locators
- Minor UI changes
- Timing issues
- Synchronization problems
- Small test maintenance fixes

This significantly reduces manual maintenance effort.

Example Prompt

Select the agent:

```
playwright-test-healer
```

Prompt:

Run all the tests until every test passes.

Output

The Healer:

- Executes the entire test suite
- Detects failing tests
- Analyzes the failure
- Repairs the test automatically
- Re-runs the tests
- Repeats the process until all tests pass (where possible)

Advantages of Playwright Test Agents

- AI-powered end-to-end test automation
- No need to manually write every test
- Automatically explores the application
- Generates structured test plans
- Creates Playwright test scripts
- Reduces test maintenance effort
- Automatically fixes common test failures
- Improves test reliability
- Saves significant development time
- Ideal for both beginners and experienced automation engineers

Summary

Playwright Test Agents automate the complete test automation lifecycle.

Agent	Responsibility	Output
Planner	Explores the application and creates a test plan	Markdown Test Plan (.plan.md)
Generator	Converts the test plan into Playwright tests	<code>.spec.ts</code> files
Healer	Executes and repairs failing tests	Stable and maintainable test suite

By combining these three agents, you can move from **application exploration** to **fully automated, self-healing Playwright tests** with minimal manual effort.